

STANDARD TEMPLATE LIBRARY

STL IN C++

Containers · Iterators · Algorithms · Functors · Real-World Projects

Containers

Iterators

Algorithms

Functors & λ

Real Projects

TABLE OF CONTENTS

01

What Is the STL?

History, philosophy, three pillars overview

02

STL Containers

vector, list, deque, set, map, unordered_map...

03

Iterators

Categories, traversal, iterator adaptors

04

Algorithms

sort, find, transform, accumulate, 80+ functions

05

Function Objects & Lambdas

Functors, predicates, lambda expressions

06

Real-World Projects

Student DB, inventory, social graph, word count

SECTION 01

What Is the STL?

Three Pillars

WHAT IS THE STANDARD TEMPLATE LIBRARY?

The **STL** (Standard Template Library) is a software library that provides a collection of **generic container classes, algorithms, and iterators**, all built on C++ templates. Introduced by **Alexander Stepanov** and incorporated into the C++98 standard.

📦 Containers

Generic data structures: vector, list, deque, set, map, unordered_map, stack, queue and more. Store any type safely.

Containers
Store data

```
vector<int> v = {3,1,4};
```

∞ Iterators

Generalized pointers that traverse containers uniformly. Decouple algorithms from data structures

Iterators
Traverse it

```
v.begin()
```

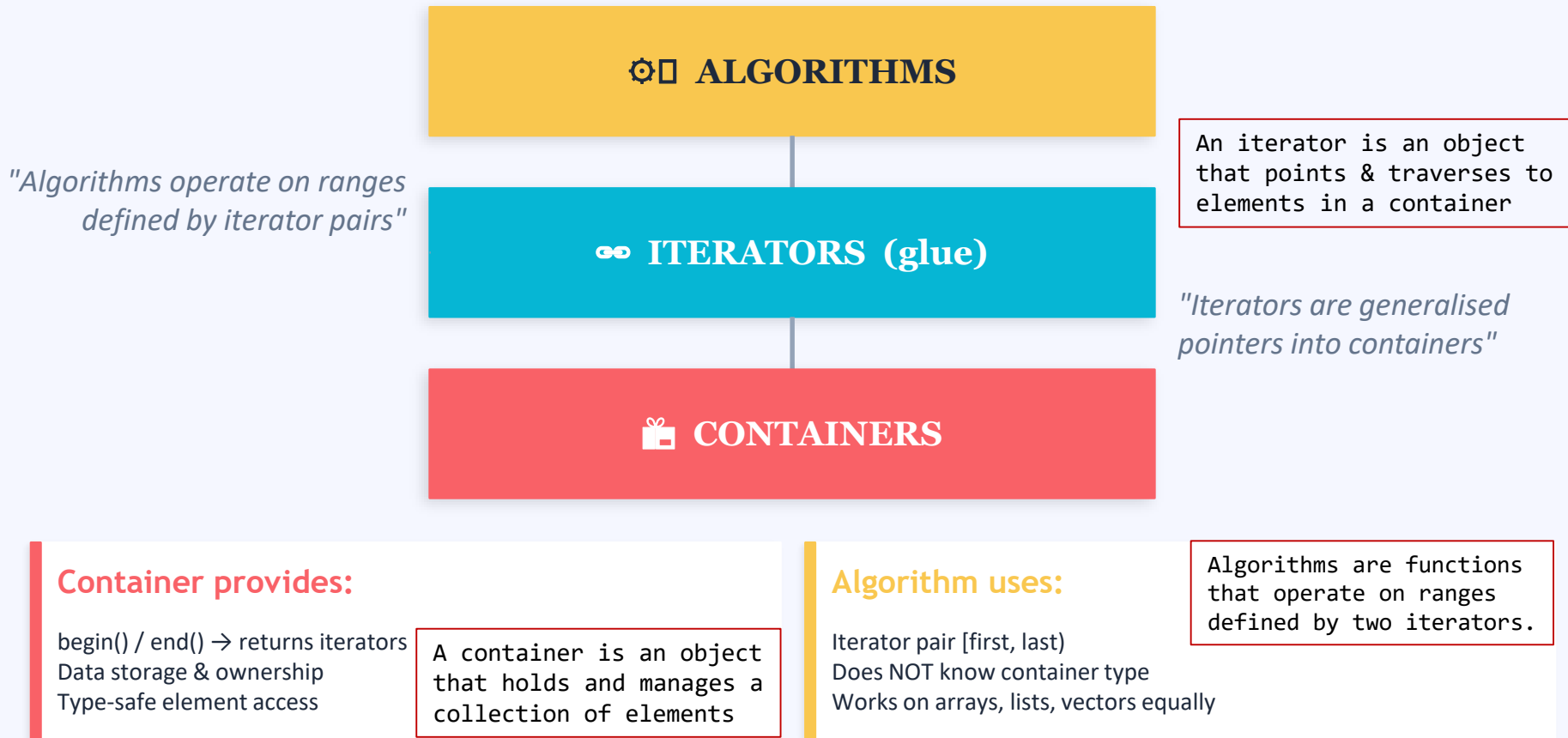
⚙️ Algorithms

Reusable functions: sort, find, count, transform, accumulate... work on any container via iterators.

Algorithms
Process it

```
sort(v.begin(), v.end())
```

STL ARCHITECTURE — HOW THE THREE PILLARS CONNECT



SECTION 02

STL Containers

CONTAINERS

- A container stores data which may consist of built-in types such as int and float, or of class objects
- The containers are divided into three major categories
 - i. Sequence containers
 - ii. Associative containers
 - iii. Container adapters
- The **sequence** containers represent **linear data structures**, such as vectors and linked lists
- **Associative** containers are **nonlinear containers** that typically can locate elements stored in the containers quickly. Such containers can store **sets of values or key/value pairs**
- Sequence and Associative containers are also called **first-class containers**
- **Container** adapters are the **constrained versions of the sequential containers** e.g. stacks and queues

STL CONTAINER CATEGORIES — OVERVIEW

Sequential Containers

`vector<T>` – dynamic array, random access $O(1)$
`deque<T>` – double-ended queue, $O(1)$ front/back
`list<T>` – doubly-linked list, $O(1)$ insert anywhere
`forward_list<T>` – singly-linked list
`array<T,N>` – fixed-size array, stack-allocated

Associative Containers (ordered)

`set<T>` – unique sorted elements
`multiset<T>` – sorted, allows duplicates
`map<K,V>` – sorted key-value pairs
`multimap<K,V>` – sorted, duplicate keys allowed

Unordered Containers (hash-based)

`unordered_set<T>` – hash set, $O(1)$ avg
`unordered_multiset<T>` – hash multiset
`unordered_map<K,V>` – hash map, $O(1)$ avg
`unordered_multimap` – hash multimap

Container Adaptors

`stack<T>` – LIFO, wraps deque
`queue<T>` – FIFO, wraps deque
`priority_queue<T>` – max-heap by default
`span<T>` – C++20 non-owning view

STL CONTAINER COMMON FEATURES

Most STL containers provide similar functionality

- Operations that apply to all containers

Functions common for all containers			
Default constructor	copy constructor	destructor	empty
insert	size	=	<
<=	>	>=	==
!=	swap		

- Operations that apply to subsets of similar containers

Functions common for all first-class containers			
max_size	begin	end	rbegin
rend	erase	clear	

std::vector — THE WORKHORSE CONTAINER

std::vector DEEP DIVE

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    // Construction
    vector<int> v1;           // empty
    vector<int> v2(5, 0);    // {0,0,0,0,0}
    // initializer list
    vector<int> v3 = {10, 20, 30, 40, 50};
    vector<string> names{"Alice","Bob",
                        "Charlie"};

    // Insertion - removal
    v3.push_back(60);        // {10,20,30,40,50, 60}
    v3.insert(v3.begin()+2,99);
    // {10,20,99,30,40,50,60}
    v3.pop_back();           // removes 60
    v3.erase(v3.begin());    // removes 10

    // Access
    cout << v3[0];          //unchecked – undefined
                             // behavior on bad index

    cout << v3.at(0);        // checked – throws
                             // std::out_of_range
    cout << v3.front();      // first element
    cout << v3.back();       // last element

    // Size & capacity
    cout << v3.size();       //current number of
                             //elements
    cout << v3.capacity();   // allocated space

    //pre-allocate – avoids reallocations
    v3.reserve(100);

    // Iteration
    for (int x : v3) cout << x << " ";
    // range-for
    for (auto it=v3.begin();it!=v3.end(); ++it)

    // iterator
    cout << *it << " ";

    for (size_t i=0; i<v3.size(); ++i)
    // index
        cout << v3[i] << " ";
}
```

vector Complexity

Operation	Time
push_back()	O(1) amort.
pop_back()	O(1)
operator[]	O(1)
insert (middle)	O(n)
erase (middle)	O(n)
find (unsorted)	O(n)
reserve()	O(n)



When to use vector:

Default container choice — cache-friendly,
fast random access, minimal overhead.

std::list AND std::deque

std::list – doubly-linked list

```
#include <list>
list<int> lst = {10, 20, 30, 40, 50};

// O(1) insert/erase anywhere (with iterator)
auto it = lst.begin();
advance(it, 2);    // move to index 2
lst.insert(it, 99); // {10,20,99,30,40,50}

lst.push_front(5); // O(1) ← key advantage
lst.push_back(60); // O(1)
lst.remove(20);    // remove all 20s
lst.sort();        // built-in sort (not STL sort!)
lst.reverse();     // reverse in place
lst.unique();      // remove consecutive duplicates

// ⚠ NO random access: lst[2] is ILLEGAL
// Must use iterators to traverse
for (auto& x : lst) cout << x << " ";

// Use list when:
// ✓ Frequent inserts/deletes in middle
// ✓ Iterators must not be invalidated
// ✗ Don't need random access
```

std::deque – double-ended queue

```
#include <deque>
deque<int> dq = {10, 20, 30};

// O(1) at BOTH ends – unlike vector
dq.push_front(5); // {5,10,20,30}
dq.push_back(40); // {5,10,20,30,40}
dq.pop_front();   // removes 5
dq.pop_back();    // removes 40

// Random access like vector: O(1)
cout << dq[1];    // 20
cout << dq.at(1); // checked access

// Not contiguous in memory
// (sequence of fixed-size chunks)

// Use deque when:
// ✓ Need push_front AND push_back O(1)
// ✓ Need random access O(1)
// ✗ Slightly slower than vector
//   due to chunk indirection
```

Container Comparison at a Glance

Container	push_back	push_front	Random Access	Middle Insert	Memory
vector	O(1) amort	O(n)	O(1)	O(n)	contiguous
deque	O(1)	O(1)	O(1)	O(n)	chunked
list	O(1)	O(1)	O(n)	O(1)	scattered

SEQUENTIAL CONTAINERS – Vector, List, Deque

VECTOR – one block, cache friendly

RAM: [1][2][3][4][5][6][7][8]

one allocation

LIST – scattered nodes, cache unfriendly

RAM: [node@0x100] [node@0x4F0] [node@0x230]

↓

↓

↓

prev/next prev/next prev/next
pointers link them together

DEQUE – chunked blocks, moderate cache use

RAM: [1][2][3] ... [4][5][6] ... [7][8][9]

chunk 1 chunk 2 chunk 3

index map links chunks together

When to use which

Situation	Best choice	Why
Default / general use	vector	Fastest cache, random access
Frequent push/pop front AND back	deque	$O(1)$ both ends
Frequent insert/delete in middle	list	$O(1)$ with iterator
Need random access	vector or deque	list has none
Implementing a queue	deque	Powers <code>std::queue</code> internally
Implementing a stack	vector	<code>push_back/pop_back</code> = $O(1)$
Lots of splicing between lists	list	<code>splice()</code> is $O(1)$
Memory is tight	vector	No per-node pointer overhead

ORDERED ASSOCIATIVE CONTAINERS — `std::map` AND `std::set`

map & set

```
// std::map — sorted key→value pairs
#include <map>
map<string,int> grades;
grades["Alice"] = 95;  grades["Bob"] = 87;  grades["Carol"] = 92;
grades.insert({"Dave", 78});

// Lookup
cout << grades["Alice"];           // 95 (creates if missing!)
cout << grades.at("Bob");          // 87 (throws if missing — safer)

if (grades.count("Eve"))           // 0 or 1 (safe check)
    cout << grades["Eve"];

// Iteration (always in sorted key order)
for (const auto& [name, grade] : grades) // C++17 structured bindings
    cout << name << ": " << grade << "\n";

// std::set — sorted unique elements
#include <set>
set<int> primes = {2,3,5,7,11,13};
primes.insert(17);
primes.erase(2);
bool found = primes.count(5);       // true
auto it = primes.find(7);           // iterator

// Set operations (via STL algorithms)
set<int> a={1,2,3,4}, b={3,4,5,6};
set<int> inter, diff;
set_intersection(a.begin(),a.end(),b.begin(),b.end(),inserter(inter, inter.begin()));
// inter = {3,4}
set_difference(a.begin(),a.end(),b.begin(),b.end(),inserter(diff, diff.begin()));
// diff = {1,2}
```

set stores keys only
map stores key-value pairs

map / set Complexity

Operation	map	set
insert	$O(\log n)$	$O(\log n)$
erase	$O(\log n)$	$O(\log n)$
find	$O(\log n)$	$O(\log n)$
count	$O(\log n)$	$O(\log n)$
iteration	$O(n)$	$O(n)$
Memory	~3× vector	~3× vector

- 💡 Use map/set when:
- Need sorted iteration
 - $O(\log n)$ lookups acceptable
 - Keys comparable with `<`
 - Use `unordered_map` for $O(1)$ avg

CONTAINER ADAPTORS — stack, queue, priority_queue

CONTAINER ADAPTORS

```
// — stack<T> — LIFO using deque (default) or vector —  
#include <stack>  
stack<int> s;  
s.push(10); s.push(20); s.push(30);  
cout << s.top();    // 30 (peek)  
s.pop();           // removes 30  
cout << s.size();   // 2  
while (!s.empty()) { cout << s.top() << " "; s.pop(); } // 20 10
```

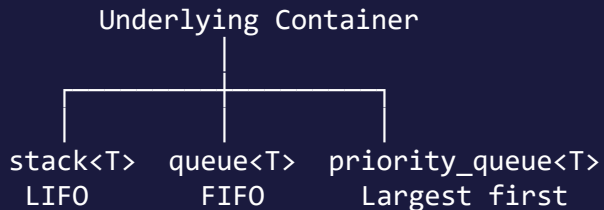
```
// Use stack with vector as underlying container  
stack<int, vector<int>> sv;
```

```
// — queue<T> — FIFO using deque —  
#include <queue>  
queue<string> q;  
q.push("Alice"); q.push("Bob"); q.push("Carol");  
cout << q.front(); // Alice (first in)  
cout << q.back();  // Carol (last in)  
q.pop();           // removes Alice  
cout << q.front(); // Bob
```

```
// — priority_queue<T> — max-heap by default —  
priority_queue<int> pq;  
pq.push(5); pq.push(1); pq.push(9); pq.push(3);  
while (!pq.empty()) { cout << pq.top() << " "; pq.pop(); } // 9 5 3 1
```

```
// Min-heap using greater<T>  
priority_queue<int, vector<int>, greater<int>> minPQ;  
minPQ.push(5); minPQ.push(1); minPQ.push(9);  
cout << minPQ.top(); // 1
```

- **Container adaptors** are wrappers around existing containers that restrict or adapt their interface to provide a specific data structure behaviour.
- They don't store data themselves — **they sit on top of an underlying container and expose only the operations that make sense for that structure.**



CHOOSING THE RIGHT CONTAINER

Quick Selection Guide

Default choice	<code>vector<T></code>
Need front insert $O(1)$	<code>deque</code> / <code>list</code>
Fast key lookup	<code>unordered_map</code>
Sorted key iteration	<code>map</code> / <code>set</code>
Unique elements	<code>set</code> / <code>unordered_set</code>
LIFO (stack)	<code>stack<T></code>
FIFO (queue)	<code>queue<T></code>
Priority / heap	<code>priority_queue<T></code>

Rule of thumb: default to `vector`. Only switch when profiling shows a bottleneck.

SECTION 03

Iterators

ITERATORS

- Iterators are pointer-like entities that are used to access individual data items (elements) in a container
- Often they are used to move sequentially from element to element, a process called iterating through the container

ITERATOR CATEGORIES

Input Iterator

Read only, single pass

ops: ++, *, !=, ==

e.g. `istream_iterator`

Output Iterator

Write only, single pass

ops: ++, * (write)

e.g. `ostream_iterator`

Forward Iterator

Read/write, multi-pass

ops: ++, *, !=, == (multi)

e.g.
`forward_list::iterator`

Bidirectional Iterator

+ go backwards

ops: ++, --, *, !=, ==

e.g. `list::iterator`

Random Access Iterator

+ arbitrary jumps $O(1)$

ops: ++, --, *, +, -, []

e.g. `vector::iterator`

Contiguous Iterator

+ contiguous guarantee C++17

ops: All + contiguous mem

e.g. `std::span`, `std::array`

ITERATORS IN PRACTICE — ADAPTORS

ITERATORS: PATTERNS & ADAPTORS

```
#include <iterator>
#include <vector>
#include <list>
#include <algorithm>
using namespace std;

vector<int> v = {5,3,8,1,9,2,7,4,6};

// — Basic iteration patterns —
for (auto it= v.begin(); it != v.end(); ++it) cout << *it << " ";
for (auto it= v.rbegin(); it != v.rend(); ++it) cout << *it << " ";
// reverse
for (auto it = v.cbegin(); it != v.cend(); ++it) cout << *it << " ";
// const

// — advance, distance, next, prev —
auto it = v.begin();
advance(it, 3);      // move 3 steps forward O(n) for list
cout << *it;         // v[3] = 1

cout << distance(v.begin(), it);    // 3

auto it2 = next(it, 2);      // iterator 2 ahead
auto it3 = prev(it, 1);      // iterator 1 behind

// — Iterator adaptors —
// back_inserter: calls push_back on insert
vector<int> dest;
copy(v.begin(), v.end(), back_inserter(dest)); //copies v into dest

// front_inserter: calls push_front (for list/deque)
list<int> lst;
copy(v.begin(), v.end(), front_inserter(lst)); //copies in reverses

// inserter: calls insert at a position
vector<int> mid = {100, 200};
copy(v.begin(), v.end(), inserter(mid, mid.begin()+1)); // insert
after 100
```

SECTION 04

STL

Algorithms

sort · find · transform · accumulate · reusable functions

ALGORITHMS

- Algorithms in the STL are procedures that are applied to containers to process their data in various ways
- For example, there are algorithms to sort, copy, search, and merge data
- Algorithms are represented by stand alone template functions
- You can also use them with built-in C++ arrays

STL ALGORITHM CATEGORIES

Non-modifying

`find, find_if, find_if_not`

`count, count_if`

`all_of, any_of, none_of`

`equal, mismatch`

`search, search_n`

Modifying

`copy, copy_if, copy_n`

`transform`

`replace, replace_if`

`fill, fill_n`

`generate, generate_n`

Sorting & Order

`sort, stable_sort`

`partial_sort, nth_element`

`binary_search`

`lower_bound, upper_bound`

`is_sorted, is_partitioned`

Numeric (#<numeric>)

`accumulate`

`inner_product`

`partial_sum`

`adjacent_difference`

`iota (fill sequence)`

Set Operations

`set_union`

`set_intersection`

`set_difference`

`set_symmetric_difference`

`includes`

Heap Operations

`make_heap`

`push_heap, pop_heap`

`sort_heap`

`is_heap, is_heap_until`

STL ALGORITHMS IN ACTION

ALGORITHMS IN ACTION

```
#include <algorithm>
#include <numeric>
#include <vector>
using namespace std;

vector<int> v = {5,3,8,1,9,2,7,4,6,3,8};

// — Sorting —
sort(v.begin(), v.end()); // ascending
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.begin(), v.end(), [](int a, int b){ //custom comparator
    return abs(a) < abs(b); });

// — Searching —
auto it = find(v.begin(), v.end(), 7); // linear O(n)
auto it2=find_if(v.begin(), v.end(), [](int x){ return x>5;});
bool exists = binary_search(v.begin(), v.end(), 8);
auto lb = lower_bound(v.begin(), v.end(), 5); // first ≥ 5

// — Counting —
int cnt = count(v.begin(), v.end(), 3); // count 3s
int big = count_if(v.begin(), v.end(), [](int x){ return x > 5;
});

// — Transform —
vector<int> squares(v.size());
transform(v.begin(), v.end(), squares.begin(),
    [](int x){ return x*x; });

// — Accumulate —
int sum = accumulate(v.begin(), v.end(), 0); // sum
int prod = accumulate(v.begin(), v.end(), 1, multiplies<int>());
string joined = accumulate(v.begin(),v.end(),string{},
    [](const string& a,int b){ return a+(a.empty()?"":",")
    +to_string(b); });

// — Remove + Erase idiom —
v.erase(remove(v.begin(),v.end(),3), v.end()); // erase all 3s
v.erase(remove_if(v.begin(),v.end(),[](int x){return
x%2==0;}),v.end());
```

SECTION 05

Functors & Lambdas

FUNCTION OBJECTS (FUNCTORS) & LAMBDA EXPRESSIONS

FUNCTORS & LAMBDA

```
// — Functor: a class with operator() —————
class MultiplyBy {
    int factor_;
    MultiplyBy(int f) : factor_(f) {}
    int operator()(int x) const { return x * factor_; }
};

vector<int> v = {1,2,3,4,5};
transform(v.begin(), v.end(), v.begin(), MultiplyBy(3));
// v = {3,6,9,12,15}

// — Built-in functors (<functional>) —————
plus<int>{}(3,4);           // 7
minus<int>{}(10,4);         // 6
multiplies<int>{}(3,4);     // 12
greater<int>{}(5,3);        // true
equal_to<string>{}("a","a"); // true

sort(v.begin(),v.end(), greater<int>()); //descending using functor

// — Lambda expressions (C++11) —————
// [captures](parameters) -> return_type { body }
auto square = [](int x) { return x*x; };
cout << square(5);    // 25

int base = 10;
```

```
//capture by value
auto addBase = [base](int x) { return x + base;};
// capture by reference
auto addBaseRef = [&base](int x) { return x + base;};

// Lambda in algorithms
auto it = find_if(v.begin(),v.end(), [](int x){ return x > 6; });
sort(v.begin(),v.end(), [](int a,int b){ return a > b; });
// descending
```

Function objects (also called functors) are objects that behave like functions — **any class that overloads operator() can be called like a function.**

A lambda is an anonymous function — **a function defined inline, right where you need it, without giving it a name.**

```
auto add = [](int a, int b) {
    return a + b;
};
add(2, 3); // 5
```

std::string AND std::string_view

std::string & string_view

```
// std::string is part of STL – fully integrated with algorithms
```

```
#include <string>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
string s = "Hello, World!";
```

```
// String operations
```

```
cout << s.size();           // 13
```

```
cout << s.length();         // 13
```

```
cout << s.substr(7, 5);      // "World"
```

```
cout << s.find("World");     // 7
```

```
cout << s.rfind('l');        // 10
```

```
s.replace(7, 5, "C++");      // "Hello, C++!"
```

```
s.insert(5, " University");  // insert at pos 5
```

```
s.erase(5, 10);             // erase 10 chars at pos 5
```

```
// Convert
```

```
int n = stoi("42");          double d = stod("3.14");
```

```
string str = to_string(99);
```

```
// STL algorithms work on strings!
```

```
sort(s.begin(), s.end());
```

```
reverse(s.begin(), s.end());
```

```
transform(s.begin(), s.end(), s.begin(), ::toupper);
```

```
int vowels = count_if(s.begin(), s.end(), [](char c){
```

```
    string v="aeiouAEIOU"; return v.find(c)!=string::npos; }));
```

```
// — C++17: string_view – non-owning, zero-copy reference —
```

```
#include <string_view>
```

```
void printFirst(string_view sv, size_t n) {
```

```
    cout << sv.substr(0, n);    // NO allocation – just a view
```

```
}
```

```
string data = "Hello World";
```

```
printFirst(data, 5);          // "Hello" – no copy of data!
```

```
printFirst("literal text", 7); // works on string literals too
```

```
// string_view is ideal for function parameters when no ownership  
needed
```

```
// Avoids converting const char* → string (which allocates memory)
```

std::pair AND std::tuple — UTILITY TYPES

pair & tuple

```
// — std::pair — two values of potentially different types —
#include <utility>
pair<string,int> person = {"Alice", 30};
cout << person.first << " is " << person.second << " years old\n";
```

```
// make_pair (pre-C++11 convenience)
auto p = make_pair(3.14, "pi");
```

```
// Pairs work directly in map (key-value = pair)
map<string,int> m;
for (const pair<const string,int>& kv : m) ...
// C++17: for (const auto& [k,v] : m) ...
```

```
// Sort vector of pairs by second element
vector<pair<string,int>> students = {{ "Ali",95},{ "Sara",98},
                                   { "Bob",87}};
sort(students.begin(), students.end(),
     [](const auto& a, const auto& b){ return a.second > b.second; });
```

```
// — std::tuple — N values of different types —
#include <tuple>
tuple<string,int,double,bool> record = {"Alice", 30, 75000.0, true};
```

```
// Access by index
cout << get<0>(record);    // Alice
cout << get<2>(record);    // 75000.0
```

```
// C++17: structured bindings (much cleaner!)
auto [name, age, salary, active] = record;
cout << name << " earns " << salary << "\n";
```

```
// make_tuple
auto t = make_tuple(1, "hello", 3.14);
```

```
// Useful for functions returning multiple values
tuple<int,string,bool> parseResult(const string& input) {
    // ... parse ...
    return {200, "OK", true};    // C++17 CTAD
}
auto [code, msg, ok] = parseResult("request");
```

STL NUMERIC ALGORITHMS — <numeric>

NUMERIC ALGORITHMS

```
#include <numeric>
#include <vector>
#include <algorithm>
using namespace std;

vector<int> v = {1,2,3,4,5,6,7,8,9,10};
vector<double> prices = {10.5, 22.3, 5.8, 18.9, 33.1};

// — accumulate — fold operation (sum, product, concat...) —
int sum = accumulate(v.begin(),v.end(), 0); // 55
int prod = accumulate(v.begin(),v.end(), 1, multiplies<int>());
// 3628800
double avg = (double)sum / v.size();

// Real use: total cart value
double total = accumulate(prices.begin(),prices.end(), 0.0);
// 90.6

// — iota — fill with increasing sequence —
vector<int> seq(10);
iota(seq.begin(), seq.end(), 0); // {0,1,2,3,4,5,6,7,8,9}
iota(seq.begin(), seq.end(), 1); // {1,2,3,4,5,6,7,8,9,10}

// — partial_sum — running totals —
vector<int> totals(v.size());
partial_sum(v.begin(),v.end(), totals.begin());

// {1,3,6,10,15,21,28,36,45,55}

// — adjacent_difference — differences between consecutive —
vector<int> diffs(v.size());
adjacent_difference(v.begin(),v.end(), diffs.begin());
// {1,1,1,1,1,1,1,1,1,1}

// — inner_product — dot product —
vector<int> a={1,2,3}, b={4,5,6};
int dot = inner_product(a.begin(),a.end(), b.begin(), 0);
// 1*4+2*5+3*6=32

// — C++17: reduce — parallel-friendly accumulate —
#include <execution>
// parallel sum
auto sum = reduce(execution::par, v.begin(), v.end());

// parallel sum of squares
auto sum = transform_reduce(execution::par, v.begin(), v.end(), 0,
plus<>{}, [](int x){ return x*x; });
```

SECTION 06

Real-World Projects

Student DB · Inventory · Word freq · Graph adjacency

PROJECT 1: STUDENT GRADE MANAGEMENT SYSTEM

STUDENT GRADE SYSTEM

```
#include <vector>
#include <algorithm>
#include <numeric>
#include <map>
using namespace std;

struct Student {
    string name;
    int id;
    vector<double> grades;
    double average() const {
        return grades.empty() ? 0.0 :
        accumulate(grades.begin(),grades.end(),0.0) / grades.size();
    }
};

int main() {
    vector<Student> roster = {
        {"Ali", 101, {88,92,79,95}},
        {"Sara", 102, {95,98,92,100}},
        {"Zain", 103, {72,68,75,80}},
        {"Maryam",104, {85,90,88,92}},
    };

    // — Sort by average descending —
    sort(roster.begin(), roster.end(),
        [](const Student& a, const Student& b){ return
        a.average() > b.average(); });

    // — Print ranking —
    for (int i=0; i<(int)roster.size(); ++i)
        printf("Rank %d: %-10s Avg: %.2f\n", i+1,
        roster[i].name.c_str(), roster[i].average());

    // — Find top student —
    auto top = max_element(roster.begin(), roster.end(),
        [](const Student& a, const Student& b){ return
        a.average() < b.average(); });
    cout << "Top: " << top->name << "\n";

    // — Count students above 85 avg —
    int honor = count_if(roster.begin(), roster.end(),
        [](const Student& s){ return s.average() >= 85.0; });
    cout << honor << " students qualify for honor roll.\n";

    // — Grade distribution map —
    map<char,int> dist;
    for (auto& s : roster) {
        double avg = s.average();
        char grade = avg>=90?'A':avg>=80?'B':avg>=70?'C':'D';
        dist[grade]++;
    }
    for (auto& [g,c] : dist) cout << g << ": " << c << "
    students\n";
}
```

PROJECT 2: PRODUCT INVENTORY MANAGEMENT

INVENTORY SYSTEM

```
#include <vector>
#include <map>
#include <algorithm>
#include <numeric>
using namespace std;

struct Product {
    int id;
    string name, category;
    double price;
    int stock;
};

class Inventory {
    vector<Product> products_;
    map<string, vector<int>> catIndex_; // category → product indices
public:
    void add(Product p) {
        catIndex_[p.category].push_back(products_.size());
        products_.push_back(move(p));
    }

    // Find by name (linear search)
    Product* find(const string& name) {
        auto it = find_if(products_.begin(), products_.end(),
            [&name](const Product& p){ return p.name == name; });
        return it != products_.end() ? &*it : nullptr;
    }

    // Products below minimum stock (reorder alert)
    vector<Product> lowStock(int minStock=10) const {
        vector<Product> result;
        copy_if(products_.begin(), products_.end(), back_inserter(result),
            [minStock](const Product& p){ return p.stock < minStock; });
        return result;
    }

    // Total value of inventory
    double totalValue() const {
        return accumulate(products_.begin(), products_.end(), 0.0,
            [](double sum, const Product& p){ return sum + p.price *
                p.stock; });
    }

    // Sort by price
    void sortByPrice(bool ascending=true) {
        sort(products_.begin(), products_.end(),
            [ascending](const Product& a, const Product& b){
                return ascending ? a.price < b.price : a.price > b.price;
            });
    }

    void printAll() const {
        for (const auto& p : products_)
            printf("%-15s $%.2f stock:%d [%s]\n",
                p.name.c_str(), p.price, p.stock, p.category.c_str());
    }
};
```

PROJECT 3: WORD FREQUENCY COUNTER

WORD FREQUENCY ANALYSER

```
#include <unordered_map>
#include <vector>
#include <algorithm>
#include <sstream>
#include <fstream>
using namespace std;

// Count word frequencies from a text file (like Google's word2vec preprocessing)
map<string,int> wordFrequency(const string& filename) {
    ifstream file(filename);
    unordered_map<string,int> freq;
    string word;
    while (file >> word) {
        // Normalize: lowercase, strip punctuation
        transform(word.begin(),word.end(),word.begin(),::tolower);

word.erase(remove_if(word.begin(),word.end(),::ispunct),word.end());
        if (!word.empty()) freq[word]++;
    }
    return {freq.begin(), freq.end()}; // convert to sorted map
}

// Top N words sorted by frequency
vector<pair<string,int>> topN(const map<string,int>& freq, int n) {
    vector<pair<string,int>> v(freq.begin(), freq.end());
    partial_sort(v.begin(), v.begin()+min(n,(int)v.size()), v.end(),
        [](const auto& a, const auto& b){ return a.second > b.second; });
    v.resize(min(n,(int)v.size()));

    return v;
}

int main() {
    auto freq = wordFrequency("hamlet.txt");

    // Top 10 words
    auto top10 = topN(freq, 10);
    cout << "=== Top 10 Words ===\n";
    for (auto& [word,cnt] : top10)
        printf("%-15s %5d times\n", word.c_str(), cnt);

    // Words appearing exactly once (hapax legomena)
    int unique = count_if(freq.begin(), freq.end(),
        [](const auto& p){ return p.second == 1; });
    cout << "\nUnique words (hapax): " << unique << "\n";

    // Vocabulary size
    cout << "Total vocabulary: " << freq.size() << " distinct words\n";

    // Search
    string query = "hamlet";
    auto it = freq.find(query);
    if (it != freq.end())
        cout << "\"" << query << "\" appears " << it->second << "
times\n";
}
```


PROJECT 4: PRIORITY TASK SCHEDULER

PRIORITY TASK SCHEDULER

```
#include <queue>
#include <vector>
#include <functional>
#include <chrono>
using namespace std;
using Clock = chrono::system_clock;

struct Task {
    int    priority;    // higher = more urgent
    int    id;
    string description;
    Clock::time_point deadline;
    function<void()> action;

    bool operator<(const Task& o) const {
        return priority < o.priority;    // min-heap by priority number
    }
};

class Scheduler {
    priority_queue<Task>    urgent_;    // sorted by priority
    map<Clock::time_point, Task>    scheduled_;    // sorted by deadline
    vector<Task>    completed_;

public:
    void submit(Task t) {
        scheduled_[t.deadline] = t;
        urgent_.push(t);
    }

    void runNext() {
        if (urgent_.empty()) { cout << "No tasks.\n"; return; }
        Task t = urgent_.top(); urgent_.pop();
        cout << "[RUN] P" << t.priority << " - " << t.description << "\n";
        if (t.action) t.action();
        completed_.push_back(t);
    }

    void runUntilDeadline() {
        auto now = Clock::now();
        while (!urgent_.empty()) {
            Task t = urgent_.top(); urgent_.pop();
            if (t.deadline <= now) {
                cout << "[OVERDUE] " << t.description << "\n";
            }
            runNext();
        }
    }

    void stats() const {
        cout << "Pending: " << urgent_.size()
              << " Completed: " << completed_.size() << "\n";
    }
};
```

STL BEST PRACTICES

✔ Prefer algorithms over raw loops

`std::transform` is safer, more expressive, and often optimised. Raw loops are error-prone.

✔ `reserve()` before bulk `push_back`

Pre-allocate capacity to avoid $O(n)$ reallocations: `v.reserve(expected_size);`

✔ `const auto&` in range-for bodies

Avoids accidental copies of potentially large objects. Use `auto&` only when mutating.

⚠ Invalidated iterators bug

`push_back` can invalidate all vector iterators (reallocation). Store indices, not iterators.

⚠ Don't use `reserve` for `map/set`

These are tree-based. Only containers with contiguous storage benefit from `reserve()`.

✗ Don't sort a list with `std::sort`

`std::sort` requires random access iterators. `list` has its own `sort()` member function.

✔ Use range-based for when possible

`for (const auto& x : v)` is cleaner, avoids off-by-one errors, and works on any iterable.

✔ Use `emplace_back` not `push_back`

`emplace_back` constructs in-place; `push_back` copies/moves. Prefer `emplace` for custom types.

✔ Erase-remove idiom for removal

`v.erase(remove_if(v.begin(), v.end(), pred), v.end());` — correct way to remove elements.

⚠ `map[]` creates default entry

`map[key]` inserts a default value if key absent. Use `count()` or `find()` to check existence.

✗ Avoid premature container switch

Profile first. `vector` is often fastest despite $O(n)$ insert, due to cache effects.

✗ Don't compare iterators cross-container

Comparing iterators from different containers is undefined behavior.

COMMON ANTI-PATTERNS & HOW TO FIX THEM

1. Manual loop instead of algorithm

✗ BAD

```
// BAD: raw loop
int sum=0;
for (int i=0; i<v.size(); ++i)
    sum += v[i] * v[i];
```

✓ FIX

```
// GOOD: algorithm + lambda
int sum = inner_product(
    v.begin(),v.end(),v.begin(),0);
// or: accumulate with square lambda
```

2. Incorrect erase during iteration

✗ BAD

```
for (auto it=v.begin();
     it!=v.end(); ++it)
    if (*it % 2 == 0)
        v.erase(it); // UB!
// invalidated it
```

✓ FIX

```
// GOOD: erase-remove idiom
v.erase(
    remove_if(v.begin(),v.end(),
        [](int x){return x%2==0;}),
    v.end());
```

3. map[] creates phantom entries

✗ BAD

```
map<string,int> m;
if (m["key"]) // BAD:
    inserts 0
    cout << "exists";
// m now has {"key":0}!
```

✓ FIX

```
// GOOD: use count or find
if (m.count("key"))
    cout << m.at("key");
// or:
auto it = m.find("key");
if (it != m.end()) cout << it->second;
```

4. Sorting a list with std::sort

✗ BAD

```
list<int> lst = {5,3,8,1};
sort(lst.begin(),lst.end()); // ✗
// compile error!
// std::sort needs random access
```

✓ FIX

```
list<int> lst = {5,3,8,1};
lst.sort(); // ✓ built-in O(n log n)
// list has its own sort() member!
```

STL QUICK REFERENCE

CHEAT SHEET

// — Container basics

```
vector<T>    v;  v.push_back(x); v.pop_back(); v[i]; v.at(i); v.size();
list<T>      l;  l.push_front(x); l.push_back(x); l.remove(x); l.sort();
map<K,V>     m;  m[k]=v; m.at(k); m.find(k); m.count(k); m.erase(k);
set<T>       s;  s.insert(x); s.erase(x); s.find(x); s.count(x);
unordered_map<K,V> um; // same interface as map, O(1) avg
stack<T>     st; st.push(x); st.top(); st.pop(); st.empty();
queue<T>      q;  q.push(x); q.front(); q.back(); q.pop();
priority_queue<T> pq; pq.push(x); pq.top(); pq.pop();
```

// — Key algorithms

```
sort(v.begin(),v.end()); // O(n log n)
sort(v.begin(),v.end(),[](auto& a,auto& b){...}); // custom comparator
auto it = find(v.begin(),v.end(),val); // O(n)
auto it = find_if(v.begin(),v.end(),pred);
bool ok = binary_search(v.begin(),v.end(),val); // O(log n), sorted!
int n = count(v.begin(),v.end(),val);
int n = count_if(v.begin(),v.end(),pred);
transform(v.begin(),v.end(),out.begin(),func);
int s = accumulate(v.begin(),v.end(),0);
v.erase(remove_if(v.begin(),v.end(),pred),v.end()); // erase-remove
```

// — Iterator utilities

```
auto it = v.begin(); // iterator to first
auto it = v.end();   // one past last
auto it = v.rbegin(); // reverse begin
```

```
advance(it, n); // move n steps
distance(a, b); // count steps between iterators
next(it,n); prev(it,n); // non-mutating move
```

// — Back/front inserters

```
copy(src.begin(),src.end(), back_inserter(dest));
copy(src.begin(),src.end(), front_inserter(dest));
```

WHERE IS STL USED IN REAL SYSTEMS?

Game Engines

vector for entity lists, unordered_map for resource lookup by name, priority_queue for event scheduling, set for collision detection broad phase.

Financial Systems

map<date,price> for time series, priority_queue for order book (bid/ask), unordered_map for position tracking.

▣ Database Engines

Sorted containers for B-tree simulation, unordered_map for hash index, stack for expression parsing.

Web Servers

unordered_map for URL routing, queue for request handling, map for HTTP headers, vector for connection pools.

▣ Machine Learning

vector<float> for tensors, unordered_map for vocabulary, sort for beam search, accumulate for mean/variance.

Cryptography

vector<uint8_t> for byte buffers, map for cipher lookup tables, bitset for flag management.

THANK YOU